

Vaibhav Gupta (vgupta1@presidio.com)

After checking the project files, I found that exception handling is used only in **three places** in this Bus Booking System project. This project looks like it was developed with AI support, because code is clean and follows common patterns. From what I understood, the project is handling exceptions in a basic but useful way.

1. Database Layer (PostgresSqlRunner.cs)

In backend, exception handling is used while working with database transactions.

- Opens database connection.
- Starts transaction.
- Runs the database operation.

If operation is successful:

- Transaction is committed using CommitAsync().

If any error happens:

- catch block executes.
- Transaction is rolled back using RollbackAsync().
- After that exception is thrown again using throw.

This is good because if database operation fails, wrong or half data will not be saved.

backend/Data/PostgresSqlRunner.cs

```
0 references
public async Task<T> WithTransactionAsync<T>(Func<NpgsqlConnection, NpgsqlTransaction, Task<T>> operation)
{
    await using var connection = new NpgsqlConnection(_connectionString);
    await connection.OpenAsync();
    await using var transaction = await connection.BeginTransactionAsync();

    try
    {
        var result = await operation(connection, transaction);
        await transaction.CommitAsync();
        return result;
    }
    catch
    {
        await transaction.RollbackAsync();
        throw;
    }
}
```

2. Authentication Service (auth.service.ts)

In frontend auth service, exception handling is used while decoding JWT token.

- Reads token.
- Splits token data.
- Decodes payload.

If token is valid:

- User email, role, and userId are returned.

If token is invalid or damaged:

- catch block handles error.
- Instead of crashing application, it returns null values.

This helps application continue running even if token has problem.

frontend/src/app/core/services/auth.service.ts

```
3 private decodeJwt(token: string | null): DecodedSession {
4   console.log('[AuthService] decodeJwt called', { hasToken: !!token });
5   if (!token) {
6     return { email: null, role: null, userId: null };
7   }
8
9   try {
10    const [, payload] = token.split('.');
11    const normalized = payload.replace(/-/g, '+').replace(/_/g, '/');
12    const decoded = JSON.parse(atob(normalized));
13
14    return {
15      email: decoded['http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name'] ?? null,
16      role: decoded['http://schemas.microsoft.com/ws/2008/06/identity/claims/role'] ?? null,
17      userId: decoded.UserId ?? null
18    };
19   } catch {
20     return { email: null, role: null, userId: null };
21   }
22 }
23 }
```

3. API Service (api.service.ts)

In frontend API service, exception handling is used while reading data from local storage.

- Gets saved trip data from local storage.
- Converts JSON string into object.

If data is correct:

- Trip data is returned.

If parsing fails or data is corrupted:

- catch block runs.
- It returns null.

This prevents application from breaking because of invalid local storage data.

frontend/src/app/core/services/api.service.ts

```
7
8  getRememberedTrip(tripId: string): SearchResult | null {
9      console.log('[ApiService] getRememberedTrip called', { tripId });
10     const raw = localStorage.getItem(SELECTED_TRIP_KEY);
11
12     if (!raw) {
13         return null;
14     }
15
16     try {
17         const parsed = JSON.parse(raw) as SearchResult;
18         return parsed.trip.id === tripId ? parsed : null;
19     } catch {
20         return null;
21     }
22 }
```

My Understanding

Overall, exception handling in this project is simple. AI-generated code seems to focus on:

- Preventing application crash.
- Returning safe values when error happens.
- Protecting database consistency using rollback.

But one thing I noticed is logging of errors is not properly done in catch blocks.

So, exception handling is present, but only in important areas like **database transaction, authentication, and local storage handling**.

There are still several places where exception handling can be improved, such as:

- Validation of city entered during trip search.
- Passenger name validation.
- Age and traveler details validation.
- Payment request validation.
- Seat selection and booking conflict handling.